

Rapport de Projet de Fin d'Études : Développement d'un
système de collecte de données en temps réel et de gestion des
alertes pour les machines en salle de réveil

Zaibi Racha

June 7, 2025

Contents

Introduction Générale

Dans un contexte médical en constante évolution, l'optimisation de la surveillance des patients en salle de réveil constitue un enjeu crucial pour assurer des soins de qualité et réactifs. Actuellement, dans de nombreux établissements de santé en Tunisie, cette surveillance repose encore largement sur des méthodes manuelles et des systèmes centralisés limités, rendant difficile une détection rapide et efficace des situations critiques. Ce constat met en lumière l'importance de moderniser les processus de suivi post-opératoire à l'aide de technologies avancées et interconnectées.

Le présent projet, réalisé au sein du SmartLab de la Faculté de Médecine de Monastir, s'inscrit dans cette démarche d'innovation. Il vise à développer un système de collecte de données en temps réel et de gestion des alertes pour les machines utilisées en salle de réveil. Ce système repose sur une approche combinant l'extraction de données via la reconnaissance optique de caractères (OCR) à l'aide de caméras pour les machines ne disposant pas d'interfaces de communication.

L'objectif principal est d'assurer une prise en charge rapide et ciblée des patients en situation critique, en réduisant les délais d'intervention grâce à une détection automatisée des alertes. En intégrant des technologies web et mobiles, ce système ambitionne d'améliorer significativement la qualité des soins post-opératoires, tout en diminuant la charge de travail manuelle et le risque d'erreurs humaines.

Ainsi, ce projet répond à un besoin concret et urgent dans le domaine de la santé en proposant une solution intelligente, adaptable et évolutive, capable d'apporter une plus grande réactivité et efficacité dans la gestion des soins en salle de réveil.

Chapter 1

Cadre Général de Projet

1.1 Présentation de l'organisme d'accueil

Le projet a été mené dans **SmartLab** de la **Faculté de médecine de Monastir (FMM)**. La Faculté de médecine de Monastir est une faculté tunisienne créée le 15 août 1980, dédiée à l'enseignement médical et à la recherche scientifique. Reconnue pour son engagement envers l'excellence, FMM a obtenu une accréditation internationale et la certification **ISO 21**, soulignant son rôle de leader dans l'éducation médicale et la recherche.

SmartLab est une **Unité de Service Commun et de Recherche (USCR)** de pointe associée à FMM, qui se concentre sur la promotion de l'innovation dans les sciences de la santé par le biais d'efforts de collaboration entre le secteur des soins de santé et les start-ups. En intégrant la recherche, la technologie et l'entrepreneuriat, SmartLab vise à faire progresser les soins médicaux et à créer des solutions adaptées aux défis de la santé moderne. Il sert de plaque tournante pour une collaboration interdisciplinaire, en tirant parti de l'expertise pour combler les lacunes entre la recherche et les applications réelles, contribuant ainsi à l'évolution d'une nouvelle ère en médecine.



Figure 1.1: Logo FMM



Figure 1.2: Logo SmartLab

1.2 Présentation de projet

Fournissez une vue d'ensemble du projet :

- **Titre** : Développement d'un système de collecte de données en temps réel et de gestion des alertes pour les machines en salle de réveil.
- **Objectif** : Améliorer les soins aux patients en assurant des réponses rapides aux situations critiques et en augmentant l'efficacité du personnel médical.
- **Portée** : Collecte de données à partir des moniteurs Mindray et des machines plus anciennes utilisant une solution basée sur caméra, gestion des alertes et notification du personnel médical.

1.3 Étude de l'existant

1.3.1 Présentation de l'étude de l'existant

Dans les hôpitaux et cliniques en Tunisie, il n'existe actuellement **aucun système de collecte de données automatisé** en salle de réveil. Le fonctionnement repose sur :

- **Un écran de surveillance centralisé** installé dans une salle administrative, permettant au personnel de suivre les signes vitaux de plusieurs patients à distance.
- **Une surveillance manuelle régulière** : Le personnel médical (infirmiers et médecins) doit se déplacer physiquement d'un patient à l'autre pour vérifier les constantes vitales, ce qui entraîne des **délais** dans la détection des situations critiques.

1.3.2 Critiques de l'existant

Manque de réactivité et d'optimisation des alertes

- Il n'existe **aucun système de gestion des alertes** priorisant les urgences en fonction de la gravité des paramètres vitaux.
- Les soignants **ne reçoivent pas d'alertes sur leurs appareils mobiles**, ce qui oblige à une surveillance constante de l'écran centralisé.
- L'absence d'un mécanisme de tri et de priorisation des alertes peut **retarder l'intervention** sur les cas critiques.

Charge de travail accrue et inefficacité opérationnelle

- La **surveillance manuelle** et les déplacements constants du personnel entraînent une **perte de temps** et augmentent la charge de travail des soignants.
- **Risque d'erreurs humaines** dans la détection des anomalies, surtout en cas de forte affluence ou de fatigue du personnel.
- L'absence de **suivi automatisé des tendances des signes vitaux** empêche une anticipation des détériorations de l'état du patient.

1.4 Solution proposée

1.4.1 Collecte de données hybride

- Solution basée sur caméra pour les machines plus anciennes, combinant **OpenCV** et **Tesseract OCR** pour extraire les données des écrans.

1.4.2 Gestion des alertes basée sur des règles

- Mise en place d'un système permettant la priorisation automatique des alertes en fonction de seuils prédéfinis.
- Envoi des notifications via e-mails et notifications mobiles grâce à **Laravel Mailing** et **Firebase Cloud Messaging (FCM)**.
- Possibilité de filtrer les alertes en fonction des horaires du personnel médical.

1.4.3 Intégration avec l'hôpital

- Connexion aux réseaux hospitaliers pour assurer une synchronisation fluide des données.
- Affichage sur une interface dédiée (web et mobile) développée avec **React**, **Laravel** et **Flutter**.

1.5 Objectifs

- Collecte de données en temps réel dans les salles de réveil.
- Livraison priorisée des alertes en fonction de seuils définis.
- Amélioration des temps de réponse pour les situations critiques.
- Automatisation et réduction des interventions manuelles dans le traitement des données et alertes.

1.6 Méthodologie adoptée

1.6.1 Choix de la méthodologie

Comparaison des méthodologies courantes

Le tableau suivant (Tableau : ??) compare les méthodologies les plus couramment utilisées dans le développement logiciel, en mettant en évidence leurs caractéristiques clés :

Caractéristiques	Scrum	Waterfall	Lean	Kanban
Adaptabilité aux changements	✓	X	X	✓
Livraisons fréquentes	✓	X	X	X
Collaboration renforcée	✓	X	X	✓
Réponse rapide aux changements	✓	X	X	X
Gestion des risques	✓	✓	✓	✓
Niveau d'implication du client	Élevé	Faible	Moyen	Moyen
Efficacité des coûts	Élevée	Moyenne	Élevée	Moyenne
Assurance qualité	Continue	Finale	Continue	Continue
Scalabilité	Moyenne	Élevée	Élevée	Moyenne
Maintenance et support	Facile	Difficile	Facile	Facile

Table 1.1: Comparaison des méthodologies de développement logiciel

Conclusion

Par conséquent, Scrum est la méthodologie la plus adaptée pour ce projet. Les principales raisons du choix de Scrum sont les suivantes :

- **Adaptabilité aux changements** : Les besoins des utilisateurs et les exigences techniques peuvent évoluer au fil du projet, et Scrum permet d'y répondre efficacement.
- **Développement itératif** : Chaque sprint permet d'obtenir un incrément fonctionnel du produit, facilitant ainsi les tests et l'amélioration continue.
- **Collaboration et transparence** : Les réunions régulières favorisent la communication entre les équipes et les parties prenantes, assurant une meilleure gestion du projet.

- **Amélioration continue** : L'analyse régulière des performances et des difficultés permet une optimisation constante du processus de développement.

1.6.2 Présentation de SCRUM

Comme présente la figure ??, Scrum est un framework agile qui repose sur des cycles de développement appelés **sprints**. Chaque sprint a une durée définie (généralement entre 2 et 4 semaines) et aboutit à un produit fonctionnel et potentiellement livrable. Le processus Scrum comprend plusieurs éléments clés :

- **Product Backlog** : Liste des fonctionnalités à développer, priorisée par le Product Owner en fonction des besoins des utilisateurs et des objectifs du projet.
- **Sprint Backlog** : Ensemble des tâches à réaliser pendant un sprint, défini par l'équipe de développement.
- **Sprint Planning** : Réunion permettant de définir les objectifs et les tâches du sprint.
- **Daily Scrum** : Brève réunion quotidienne de l'équipe pour suivre l'avancement et identifier les obstacles.
- **Sprint Review** : Présentation de l'incrément produit aux parties prenantes à la fin de chaque sprint.
- **Sprint Retrospective** : Analyse du sprint pour identifier les axes d'amélioration.



Figure 1.3: Processus Scrum

1.6.3 Les rôles SCRUM

Dans Scrum, trois rôles principaux sont définis :

- **Product Owner** : Représentant des parties prenantes, il définit et priorise les fonctionnalités à développer en fonction des besoins des utilisateurs et des objectifs du projet.
- **Scrum Master** : Responsable de la bonne application du framework Scrum. Il facilite la communication, aide à surmonter les obstacles et veille à l'amélioration continue des processus.
- **Équipe de développement** : Composée de développeurs, designers et autres spécialistes, elle est responsable de la mise en œuvre des fonctionnalités et de la livraison des incréments du produit.

En adoptant Scrum, le projet bénéficie d'une approche structurée et efficace, garantissant un suivi rigoureux et une réponse rapide aux exigences changeantes du domaine médical.

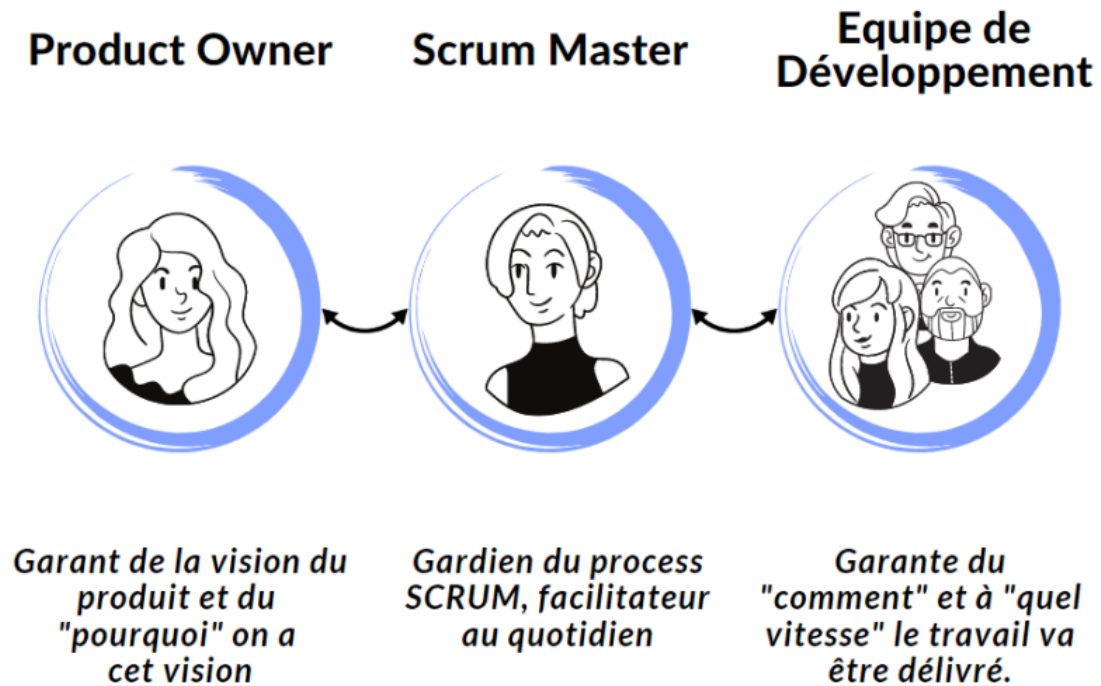


Figure 1.4: Rôles Scrum

Chapter 2

Sprint 0

2.1 Introduction

Ce chapitre présente le **Sprint 0**, qui marque la phase initiale de la réalisation de notre projet. Le **Sprint 0** est une étape cruciale où nous posons les fondations du projet en identifiant les acteurs clés, en capturant les besoins fonctionnels et non fonctionnels, et en définissant la méthodologie de travail qui guidera le développement. Cette phase permet de clarifier les objectifs, les attentes et les contraintes du projet, tout en établissant un cadre de travail structuré pour les étapes ultérieures.

2.2 Capture des besoins

2.2.1 Identification des acteurs

Dans un premier temps, nous identifions les acteurs du système, c'est-à-dire les entités externes qui interagiront avec l'application. Ces acteurs jouent un rôle central dans la définition des fonctionnalités et des services que le système doit fournir.

Acteur	Description
Médecin	Consulte les données des patients, reçoit les alertes critiques et valide les actions nécessaires.
Infirmier(ère)	Surveille les patients, reçoit les alertes en fonction de ses horaires et prend des mesures immédiates.
Administrateur du Système	Gère les utilisateurs, configure les seuils d'alerte et assure le bon fonctionnement du système.
Moniteur de Signes Vitaux	Source de données : envoie des informations vitales via API ou caméra.
Caméra	Capturent les écrans des machines et les transmet pour analyse.

Table 2.1: Acteurs du système et leurs rôles

2.2.2 Besoins fonctionnels et non fonctionnels

Cette section détaille les besoins fonctionnels et non fonctionnels du système, qui reflètent les exigences des utilisateurs ainsi que les contraintes techniques à respecter.

Besoins Fonctionnels

Les besoins fonctionnels décrivent les fonctionnalités attendues du système :

- **Collecte des données** : Le système doit être capable de collecter des données en temps réel à partir des moniteurs de signes vitaux et des caméras.
- **Détection et reconnaissance des valeurs vitales** : Utilisation de l'OCR pour extraire les valeurs vitales à partir des images capturées.
- **Génération et gestion des alertes** : Le système doit déclencher des alertes en fonction des seuils définis et les acheminer au personnel médical concerné.
- **Affichage des données** : Une interface web et mobile permettant aux médecins et infirmiers d'accéder aux données patient.
- **Authentification et gestion des utilisateurs** : Gestion des rôles et permissions des utilisateurs (administrateurs, médecins, infirmiers).
- **Notifications** : Envoi d'alertes par e-mail, SMS ou notifications push via Firebase Cloud Messaging (FCM).
- **Stockage et historique des données** : Enregistrement des valeurs mesurées et des alertes pour analyse et suivi.

Besoins Non Fonctionnels

Les besoins non fonctionnels définissent les contraintes techniques et de performance du système :

- **Disponibilité** : Le système doit être accessible 24/7 avec une tolérance aux pannes minimale.
- **Sécurité** : Protection des données médicales via chiffrement et authentification sécurisée.
- **Interopérabilité** : Compatibilité avec différents modèles de moniteurs (Mindray, Edan).
- **Scalabilité** : Capacité à gérer une augmentation du nombre de patients et d'appareils connectés.
- **Temps de réponse** : Le traitement des alertes doit être instantané pour garantir la réactivité du personnel médical.
- **Conformité** : Respect des réglementations médicales.

2.3 Pilotage du projet avec Scrum

Le projet sera géré selon la méthodologie **Scrum**, favorisant une approche agile avec des cycles itératifs appelés **sprints**. Le **Product Owner**, l'**équipe de développement** et le **Scrum Master** collaboreront pour assurer une livraison progressive des fonctionnalités clés.

Acteurs Scrum et Missions

Rôle	Mission	Acteur
Product Owner	- Définir et prioriser les fonctionnalités du produit (Product Backlog). - Assurer que l'équipe travaille sur les fonctionnalités les plus importantes. - Valider les livrables à la fin de chaque sprint.	Pr Ammeri Charfeddine
Équipe de Développement	- Concevoir, développer et tester les fonctionnalités du produit. - Participer aux réunions de planification et de revue de sprint. - Assurer la qualité du code et des livrables.	Zaibi Racha

Rôle	Mission	Acteur
Scrum Master	- Faciliter les réunions Scrum (Daily Stand-up, Sprint Planning, Sprint Review, Retrospective). - Supprimer les obstacles qui empêchent l'équipe de progresser. - Assurer que l'équipe suit les principes Scrum.	Ouni Ghada

Table 2.2: Acteurs du projet

Organisation du Backlog

Le **Product Backlog** regroupe l'ensemble des fonctionnalités à développer pour le système. Chacune est formulée sous forme de *User Story*, garantissant une approche centrée sur l'utilisateur.

Chaque fonctionnalité sera développée et livrée en plusieurs **sprints**, selon les priorités établies par le *Product Owner*. Elles sont classées selon la méthode **MoSCoW**, qui définit les niveaux de priorité pour chaque tâche.

Backlog du Projet

ID	Tâche	Priority
M-01	Détection et reconnaissance des valeurs vitales via OCR pour les machines anciennes.	M
M-02	Génération et gestion des alertes en fonction des seuils prédéfinis.	M
M-03	Envoi d'alertes aux médecins et infirmiers via notifications push (FCM) et e-mail.	M
M-04	Interface web et mobile pour visualiser les données des patients et gérer les alertes.	M
M-05	Authentification et gestion des utilisateurs (médecins, infirmiers, administrateurs).	M
M-06	Stockage et historique des données pour analyse et suivi.	M
M-07	Conformité aux réglementations médicales (sécurité des données, chiffrement).	M
M-08	Tests de validation de la collecte de données (comparaison entre API et OCR).	M
S-01	Intégration avec d'autres modèles de moniteurs (Edan, etc.).	S
S-02	Configuration des seuils d'alerte personnalisés par patient ou par bloc.	S
S-03	Gestion des horaires de filtrage des alertes pour le personnel médical.	S
S-04	Interface d'administration pour configurer les seuils d'alerte et gérer les utilisateurs.	S
S-05	Notifications SMS en plus des notifications push et e-mail.	S
S-06	Scalabilité du système pour gérer un nombre croissant de patients et d'appareils.	S

ID	Tâche	Priority
C-02	Analyse prédictive des données pour anticiper les alertes critiques.	C
C-03	Interface mobile avancée avec des graphiques interactifs pour les données des patients.	C
C-04	Support multilingue pour l'interface utilisateur.	C
C-05	Gestion des alertes par priorité (critique, moyen, faible).	C
C-06	Historique des alertes avec possibilité de filtrage par date, patient, ou type d'alerte.	C

Table 2.3: Backlog du Projet avec Priorités MoSCoW

Clé des Priorités MoSCoW

- **M (Must)** : Tâches critiques nécessaires au bon fonctionnement du système.
- **S (Should)** : Tâches importantes qui apportent une valeur significative mais ne sont pas essentielles pour la première version.
- **C (Could)** : Fonctionnalités optionnelles pouvant être ajoutées si le temps et les ressources le permettent.
- **W (Won't Have)** : Tâches hors du périmètre de la phase actuelle du projet.

2.3.1 Diagramme des cas d'utilisation global

Montrez les prototypes initiaux pour les interfaces du système (par exemple, visualisation des données, gestion des alertes).

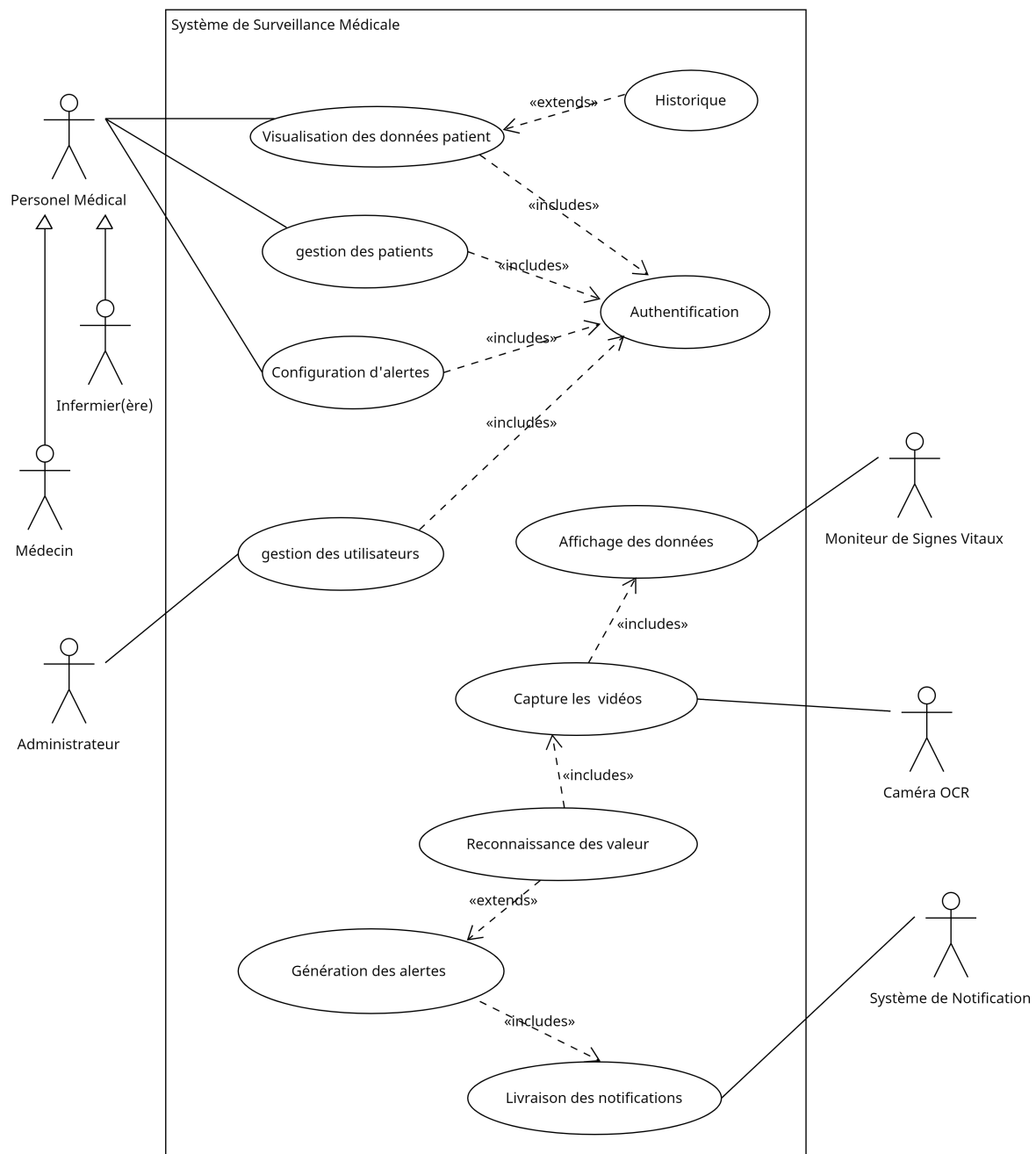


Figure 2.1: Diagramme des cas d'utilisation global

2.3.2 Planification des sprints

Une fois le backlog du produit finalisé, nous avons organisé la réunion de planification de sprint. L'objectif de cette réunion était de construire le backlog de sprint en se basant sur le backlog de produit défini par le Product Owner. À l'issue de cette réunion, nous avons identifié les durées prévisionnelles du travail à effectuer pour chaque sprint. Pour ce projet, nous avons divisé le travail en deux releases. La première release contient un seul sprint d'une durée de 2 semaines, tandis que la deuxième release est composée de deux sprints, chacun d'une durée de 3 semaines. La figure 2.2 illustre la répartition des sprints pour notre système.

Plan des sprints :

- **Sprint 1 : Collecte de données basée sur caméra**

- **Objectif** : Mettre en place un système de collecte de données à partir des machines anciennes en utilisant des caméras et des techniques de reconnaissance optique de caractères (OCR).
 - **Deliverables** :
 - * Intégration de la caméra avec le système.
 - * Détection et extraction des valeurs vitales via OCR.
 - * Validation des données collectées par rapport aux données réelles.
 - **Durée** : 2 semaines.
 - **Tâches clés** :
 - * Configuration des caméras et des logiciels OCR (OpenCV, Tesseract).
 - * Développement du pipeline de traitement d'images.
 - * Tests de validation des données collectées.
 - **Risques** :
 - * Mauvaise qualité d'image affectant la précision de l'OCR.
 - * Solution : Utilisation de caméras haute résolution et prétraitement des images.
- **Sprint 2 : Système de gestion des alertes**
 - **Objectif** : Développer un système de gestion des alertes basé sur des seuils prédéfinis, avec un affichage dans l'interface utilisateur.
 - **Deliverables** :
 - * Définition des seuils d'alerte pour les valeurs vitales.
 - * Mise en place d'un système de priorisation des alertes.
 - * Intégration d'un tableau de bord d'alertes accessible au personnel médical.
 - **Durée** : 3 semaines.
 - **Tâches clés** :
 - * Développement du module de gestion des alertes.
 - * Conception d'un affichage clair et ergonomique des alertes.
 - * Tests de validation des alertes.
 - **Dépendances** : Les données collectées dans le Sprint 1 doivent être disponibles pour ce sprint.
 - **Risques** :
 - * Retards dans la visualisation des alertes en temps réel.
 - * Solution : Optimisation du traitement des données et du rafraîchissement de l'interface.

2.3.3 Prototypage des interfaces

Montrez les prototypes initiaux pour les interfaces du système (par exemple, visualisation des données, gestion des alertes).

2.4 Environnement de travail

2.4.1 Environnement matériel

Pour assurer le bon fonctionnement du système, les exigences matérielles suivantes doivent être respectées:

- **Caméras pour les machines plus anciennes** :
 - **Type** : Caméras USB haute résolution (ex. Logitech C920).
 - **Rôle** : Capture des écrans des machines anciennes pour l'extraction des données via OCR (Reconnaissance Optique de Caractères).
 - **Configuration** : Montage fixe sur les moniteurs pour une capture stable et précise.
- **Serveurs pour le stockage et le traitement des données** :

- **Type** : Serveurs dédiés ou cloud (ex. AWS, Azure, ou serveurs locaux).
- **Rôle** : Stockage des données collectées, traitement en temps réel, et génération des alertes.
- **Spécifications techniques** :
 - * **Processeur** : Multi-core (ex. Intel Xeon ou AMD EPYC).
 - * **Mémoire RAM** : 16 Go minimum (32 Go recommandé pour une meilleure performance).
 - * **Stockage** : Disque SSD de 500 Go minimum pour un accès rapide aux données.
 - * **Connectivité** : Réseau haut débit (1 Gbps) pour une transmission rapide des données.
- **Réseau de communication** :
 - **Type** : Réseau local (LAN) sécurisé avec accès à Internet.
 - **Rôle** : Assurer la communication entre les caméras, les serveurs et les applications mobiles/web.
 - **Exigences** :
 - * **Bande passante** : 100 Mbps minimum pour une transmission fluide des données.
 - * **Sécurité** : Chiffrement des données (ex. SSL/TLS) pour protéger les informations médicales sensibles.
- **Appareils mobiles pour le personnel médical** :
 - **Type** : Smartphones et tablettes compatibles Android et iOS.
 - **Rôle** : Recevoir des notifications push en temps réel et accéder à l'interface mobile du système.
 - **Exigences** :
 - * **Version minimale** : Android 8.0 (Oreo) ou iOS 12.
 - * **Connexion** : Wi-Fi ou 4G/5G pour une réception rapide des alertes.

2.4.2 Environnement de développement

Pour le développement du système, les outils suivants seront utilisés :

- **Langages de programmation** :
 - **Python** :
 - * Version : Python 3.9 ou supérieur.
 - * Rôle : Utilisé pour le traitement des données, la collecte à partir des images via OCR, et l'intégration des bibliothèques de vision par ordinateur (OpenCV, Tesseract OCR).
 - * Frameworks : Flask ou Django pour le développement backend.
 - **Laravel** :
 - * Version : Laravel 8.x ou supérieur.
 - * Rôle : Utilisé pour le développement du backend, la gestion des utilisateurs, et l'intégration des services de notification (e-mail, FCM).
 - * Fonctionnalités clés : ORM (Eloquent), gestion des routes, et système de templates (Blade).
 - **React** :
 - * Version : React 17.x ou supérieur.
 - * Rôle : Utilisé pour le développement de l'interface web, permettant une visualisation interactive des données des patients et la gestion des alertes.
 - * Bibliothèques associées : Redux pour la gestion de l'état, Axios pour les requêtes HTTP.
 - **Flutter** :
 - * Version : Flutter 2.x ou supérieur.
 - * Rôle : Utilisé pour le développement de l'application mobile, permettant au personnel médical de recevoir des notifications et d'accéder aux données des patients en temps réel.
 - * Fonctionnalités clés : Interface native pour Android et iOS, intégration avec Firebase pour les notifications push.

- **Bibliothèques et outils :**

- **OpenCV :**

- * Version : OpenCV 4.5 ou supérieur.
 - * Rôle : Utilisé pour le traitement d'images et la détection des régions d'intérêt (ROI) dans les moniteurs de signes vitaux.
 - * Fonctionnalités clés : Manipulation d'images, détection de texte, et prétraitement des données pour l'OCR.

- **Tesseract OCR :**

- * Version : Tesseract 5.0 ou supérieur.
 - * Rôle : Utilisé pour la reconnaissance optique de caractères (OCR) afin d'extraire les valeurs vitales des images capturées par les caméras.
 - * Fonctionnalités clés : Support multilingue, haute précision pour les chiffres et les textes médicaux.

- **Firebase Cloud Messaging (FCM) :**

- * Rôle : Utilisé pour envoyer des notifications push en temps réel aux appareils mobiles du personnel médical.
 - * Fonctionnalités clés : Intégration avec Flutter et Laravel, support pour Android et iOS.

- **Docker :**

- * Rôle : Utilisé pour la conteneurisation des applications, permettant un déploiement facile et une gestion des dépendances.
 - * Fonctionnalités clés : Isolation des environnements, déploiement sur différents serveurs.

- **Git :**

- * Rôle : Utilisé pour le contrôle de version et la collaboration entre les membres de l'équipe de développement.
 - * Plateforme : GitHub ou GitLab pour le stockage des dépôts et la gestion des branches.

2.4.3 Environnement logiciel

Pour assurer le bon fonctionnement du système, les exigences logicielles suivantes doivent être respectées :

- **Systèmes d'exploitation :**

- **Serveurs :**

- * Linux : Distributions recommandées (Ubuntu Server 20.04 LTS, CentOS 8).
 - * Rôle : Hébergement des serveurs backend, gestion des bases de données, et traitement des données en temps réel.
 - * Configuration minimale : 4 cœurs CPU, 8 Go de RAM, 50 Go de stockage.

- **Postes de développement :**

- * Windows 11.
 - * Rôle : Environnement de développement (backend & frontend & mobile).
 - * Configuration minimale : 16 Go de RAM, processeur Intel i5.

- **Appareils mobiles :**

- * Android : Version 8.0 ou supérieure.
 - * Rôle : Exécution de l'application mobile pour le personnel médical.

- **Systèmes de base de données :**

- **Base de données principale :**

- * MySQL : Version 8.0 ou supérieure.
 - * Rôle : Stockage des données des patients, des alertes, et des historiques.
 - * Configuration : Base de données relationnelle avec support des transactions ACID.

- **Protocoles de communication :**

- **API REST :**

- * Rôle : Interface de communication entre les différents composants du système (frontend, backend, mobile).
- * Sécurité : Utilisation de tokens JWT (JSON Web Tokens) pour l'authentification et l'autorisation.
- * Documentation : Swagger/OpenAPI pour une documentation interactive des API.

- **WebSockets :**

- * Rôle : Communication en temps réel entre le backend et les applications mobiles/web pour les notifications push.
- * Utilisation : Envoi instantané des alertes critiques au personnel médical.

- **Autres logiciels et outils :**

- **IDE (Environnements de développement intégrés) :**

- * Visual Studio Code : Pour le développement en Python, React, et Flutter.
- * PhpStorm : Pour le développement en Laravel.

- **Gestion des dépendances :**

- * Composer : Pour la gestion des packages PHP (Laravel).
- * npm : Pour la gestion des packages JavaScript (React).
- * pip : Pour la gestion des packages Python.

- **Outils de tests :**

- * Postman : Pour tester les API REST.
- * Jest : Pour les tests unitaires en React.
- * PHPUnit : Pour les tests unitaires en Laravel.

2.5 Architecture générale de l'application

2.5.1 Architecture physique

L'architecture physique du système repose sur plusieurs composants matériels interconnectés via un réseau sécurisé. Les moniteurs Mindray et les caméras collectent les données en temps réel, qui sont ensuite transmises au système central via le réseau de surveillance interne. Les serveurs, hébergés localement ou dans le cloud, stockent et traitent les données, tandis que les appareils mobiles permettent au personnel médical de recevoir des notifications et d'accéder aux données des patients.

HEAD

L'architecture physique du système repose sur plusieurs composants matériels interconnectés via un réseau sécurisé. Les moniteurs Mindray et les caméras collectent les données en temps réel, qui sont ensuite transmises au système central via le réseau de surveillance interne. Les serveurs, hébergés localement ou dans le cloud, stockent et traitent les données, tandis que les appareils mobiles permettent au personnel médical de recevoir des notifications et d'accéder aux données des patients.

- **Moniteurs Mindray :** Collectent les données des patients en temps réel (fréquence cardiaque, pression artérielle, saturation en oxygène, etc.) et les transmettent via des interfaces réseau ou des APIs propriétaires. =====

- **Moniteurs Mindray :** Collectent les données des patients en temps réel (fréquence cardiaque, pression artérielle, saturation en oxygène, etc.) et les transmettent via des API ou des protocoles HL7. `lllllll a485d22827129f93ec9986d45273b7e0e7f02d91`
- **Caméras :** Capturent les écrans des machines et transmettent les images au service de reconnaissance de texte (OCR).
- **Serveurs :** Stockent et traitent les données collectées. Ils peuvent être hébergés localement ou dans le cloud (AWS, Azure).
- **Réseau de Communication :** Assure la transmission des données entre les moniteurs, les caméras, les serveurs et les applications mobiles/web.
- **Appareils Mobiles :** Permettent au personnel médical de recevoir des notifications push et d'accéder aux données des patients en temps réel.

2.5.2 Fonctionnement de l'Architecture

Le système fonctionne en trois étapes principales :

1. Collecte des données :

- Les données sont collectées à partir des moniteurs Mindray via des interfaces réseau ou des API propriétaires.
- Les données des machines anciennes sont capturées via des caméras et traitées pour extraire les valeurs vitales à l'aide de l'OCR.

2. Traitement des données :

- Les données sont analysées en temps réel pour détecter des anomalies ou des valeurs critiques.
- Les données sont stockées dans une base de données MySQL pour une analyse ultérieure.

3. Génération des alertes :

- Les alertes sont générées en fonction des seuils prédéfinis.
- Les alertes sont envoyées au personnel médical via des notifications push (FCM), des e-mails ou des SMS.

2.5.3 Spécification logicielle du système

Le système est développé en utilisant les technologies suivantes :

– Backend :

- * **Laravel** : Pour la gestion des utilisateurs, des données et l'intégration des services de notification (e-mail, FCM).
- * **Python** : Pour le traitement des données, l'intégration OCR (OpenCV, Tesseract), et la communication avec les services de capture et d'analyse d'image.

– Frontend :

- * **React** : Pour l'interface web, permettant une visualisation interactive des données des patients et la gestion des alertes.
- * **Flutter** : Pour l'application mobile, permettant au personnel médical de recevoir des notifications et d'accéder aux données des patients en temps réel.

– Base de données :

- * **MySQL** : Pour le stockage des données des patients, des alertes et des historiques.

– Protocoles de communication :

- * **REST API** : Pour la communication entre le frontend, le backend et l'application mobile.
- * **WebSockets** : Pour les notifications en temps réel.

– Autres outils :

- * **Docker** : Pour la conteneurisation et le déploiement facile de l'application.
- * **Git** : Pour le contrôle de version et la collaboration entre les membres de l'équipe.
- * **Postman** : Pour tester les API REST.
- * **Jest/PHPUnit** : Pour les tests unitaires en React et Laravel.

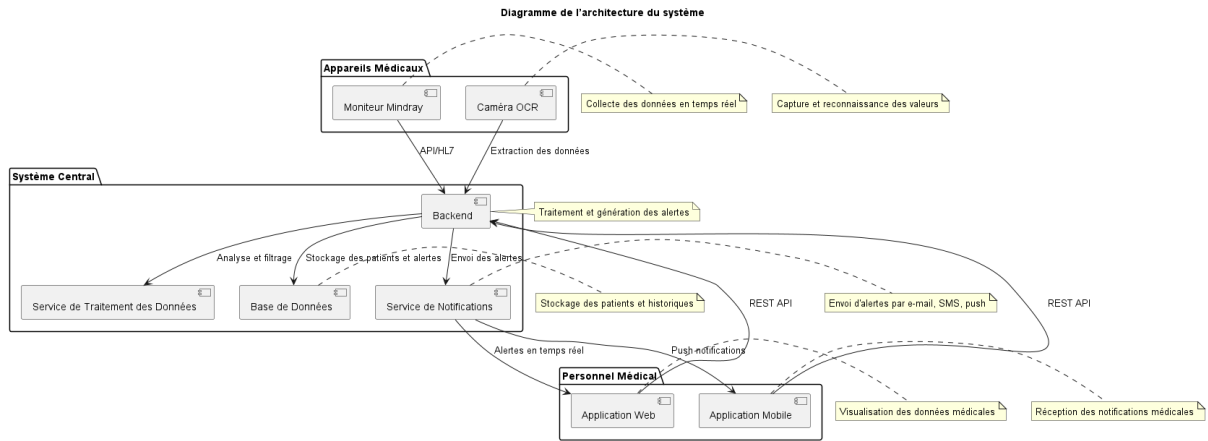


Figure 2.2: Diagramme de l'architecture du système

2.5.4 Diagramme de déploiement

Le diagramme de déploiement (Figure ??) illustre l'architecture du système, reliant les équipements médicaux, le backend et les interfaces utilisateur.

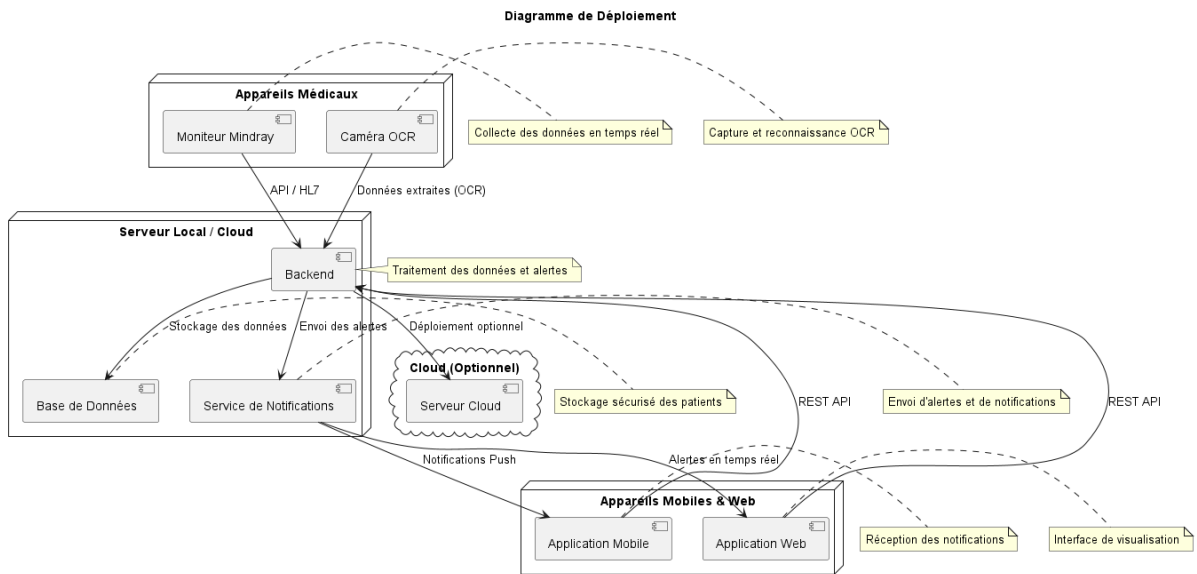


Figure 2.3: Diagramme de Déploiement du Système

2.5.5 Composants et Déploiement

Le système est composé des éléments suivants :

- **Serveur Local / Cloud :**
 - * **Backend :** Traitement des données et gestion des alertes.
 - * **Base de Données :** Stockage des données des patients et des alertes.
 - * **Service de Notifications :** Envoi d'alertes (e-mails, notifications push).
- **Appareils Médicaux :**
 - * **Moniteur Mindray :** Envoie les données vers le système central via un module d'intégration personnalisé.

- * **Caméra OCR** : Capture et extrait les valeurs vitales affichées sur les écrans des machines.
- **Applications Mobiles & Web** :
 - * **Application Web / Mobile** : Consultation des données des patients et réception des alertes via REST API.

2.5.6 Flux de Communication

Le flux de communication du système débute par la **collecte des données** à partir des moniteurs HealForce ou des machines anciennes (via OCR). Ces données sont ensuite **transmises au backend** via le réseau de surveillance interne. Le backend **analyse les données en temps réel** et les stocke dans une base de données MySQL. En cas de détection d'anomalies (par rapport aux seuils prédéfinis), le système **envoie des alertes** au personnel médical via e-mail, SMS ou notifications push (FCM). Le personnel médical peut alors **consulter les alertes et les données** via l'application web ou mobile, et **prendre les mesures nécessaires** pour répondre aux situations critiques.

Ce flux, illustré dans la figure ??, assure une communication fluide et efficace entre les appareils médicaux, le système central et le personnel médical.

2.5.7 Justification

L'architecture garantit :

- **Modularité** : Séparation claire des composants (collecte, traitement, notification).
- **Réactivité** : Alertes en temps réel pour une réponse rapide aux situations critiques.
- **Interopérabilité** : Compatibilité avec les technologies OCR et les standards d'échange de données médicales.
- **Flexibilité** : Option de déploiement sur serveur local ou cloud pour une adaptation aux besoins de l'hôpital.

Chapter 3

Étude et réalisation du Sprint 1

3.1 Backlog du sprint

Objectif : Implémenter la collecte de données (caméra + OCR) et un système de notification basique. Voir le tableau ?? pour les détails.

ID	User Story	Tâche technique	Priorité	Estimation
M-02	Détection des valeurs via OCR	Intégration de Tesseract pour l'extraction de texte	Haute	5 jours
		Utilisation d'OpenCV pour le prétraitement des images		
M-03	Génération d'alertes	Implémentation des seuils critiques	Haute	3 jours
		Développement d'un moteur de règles pour les alertes		
M-04	Envoi de notifications	Intégration de Firebase Cloud Messaging (FCM)	Moyenne	4 jours
		Configuration du service SMTP pour les e-mails		
M-09	Validation des données	Vérification des valeurs extraites avec des seuils	Moyenne	4 jours
		Correction automatique des erreurs OCR		

Table 3.1: Backlog du sprint 1

3.2 Spécification fonctionnelle

La spécification fonctionnelle décrit les fonctionnalités attendues du système. Elle détaille les interactions entre les utilisateurs (acteurs) et le système, ainsi que les processus permettant d'atteindre les objectifs définis dans le cadre du projet. Cette spécification permet de formaliser les exigences, les flux de données et les interactions entre les différents composants du système.

3.2.1 Diagramme de cas d'utilisation

La figure ?? illustre le diagramme de cas d'utilisation pour le Sprint 1, intitulé *Capture et Reconnaissance Texte*. Ce sprint se concentre sur la capture des données à partir d'un moniteur de

signes vitaux, leur reconnaissance via une caméra classique et un processus OCR, et la génération d'alertes en fonction des valeurs reconnues.

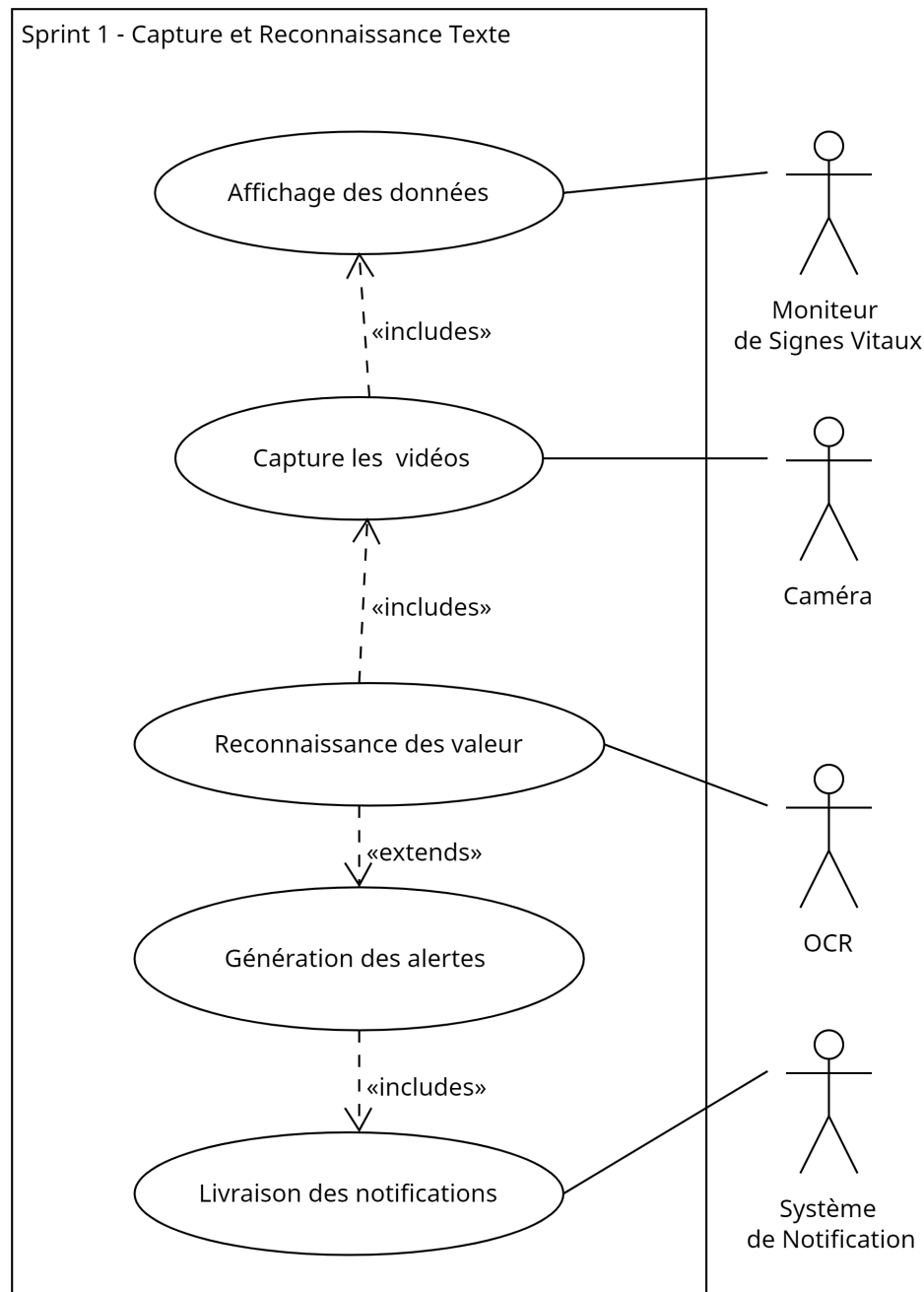


Figure 3.1: Diagramme de Cas d'Utilisation - Sprint 1

3.2.2 Acteurs Principaux

- **Moniteur de Signes Vitaux** : Fournit les données à afficher.
- **Caméra** : Capture les images ou vidéos du moniteur de signes vitaux.
- **OCR (Reconnaissance Optique de Caractères)** : Analyse les images capturées pour extraire les valeurs numériques ou textuelles.
- **Système de Notification** : Reçoit et livre les notifications générées.

3.2.3 Cas d'Utilisation

- **Affichage des données** : Permet de visualiser les données provenant du moniteur.
- **Capture d'images** : La caméra capture les images du moniteur de signes vitaux.
- **Reconnaissance des valeurs** : L'OCR analyse les images capturées pour extraire les valeurs à partir du texte.
- **Génération des alertes** : Crée des alertes basées sur les valeurs extraites par l'OCR.
- **Livraison des notifications** : Envoie les alertes générées au système de notification.

3.2.4 Description des cas d'utilisation

Affichage des données Le moniteur de signes vitaux affiche les données en temps réel. Ces données sont capturées par la caméra OCR.

Reconnaissance des valeurs La caméra OCR analyse les données capturées et extrait les valeurs numériques à l'aide de Tesseract OCR.

Génération des alertes Le système compare les valeurs reconnues avec des seuils prédéfinis. Si une valeur dépasse un seuil, une alerte est générée.

Livraison des notifications Les alertes générées sont transmises au système de notification, qui les envoie aux utilisateurs concernés.

3.3 Conception

3.3.1 Architecture technique

- Stack logicielle : Python 3.10, OpenCV 4.8, Tesseract 5.3.1
- Infrastructure : Docker containers sur Ubuntu 22.04
- Communication inter-modules : RabbitMQ

3.3.2 Diagramme de séquence

3.3.3 Composants développés

Table 3.2: Composants clés du Sprint 1

Composant	Fonctionnalités
CaptureModule	Gestion de la caméra, réglages automatiques
OCRProcessor	Pipeline de traitement d'images et reconnaissance
Composant	Fonctionnalités
CaptureModule	Gestion de la caméra, réglages automatiques
OCRProcessor	Pipeline de traitement d'images et reconnaissance
AlertEngine	Évaluation des seuils et déclenchement d'alertes
Notifier	Envoi multi-canaux (FCM, Email)

3.4 Validation

3.4.1 Stratégie de test

- Tests unitaires : Couverture > 80% (pytest-cov)

- Tests d'intégration : Jenkins pipeline
- Tests de performance : JMeter pour la latence

Chapter 4

Étude et réalisation du Sprint 2

4.1 Backlog du sprint

Définissez les tâches pour le Sprint 2 :

Chapter 5

Étude et réalisation du Sprint 3

5.1 Backlog du sprint

Définissez les tâches pour le Sprint 3 :

- Système de gestion des alertes.
- Livraison des notifications (e-mail, notifications dans l’application).

5.2 Spécification fonctionnelle

5.2.1 Diagramme de cas d’utilisation

Fournissez un diagramme des cas d’utilisation pour le Sprint 3.

5.2.2 Description textuelle de cas d’utilisation

Décrivez les cas d’utilisation en détail.

5.3 Conception

5.3.1 Diagrammes des séquences détaillés

Incluez des diagrammes de séquence pour le processus de gestion des alertes.

5.3.2 Diagramme de Classes

Fournissez un diagramme de classes pour les composants du système.

5.4 Réalisation

5.4.1 Interface de Consultation des détails de colis avec son état courant

(Adaptez cette section à votre projet si nécessaire.)

5.4.2 Interface de Consultation les événements et les états

(Adaptez cette section à votre projet si nécessaire.)

5.4.3 Interface d'ajout une action

(Adaptez cette section à votre projet si nécessaire.)

5.4.4 Interface d'envoi un mail au client

Montrez l'interface pour l'envoi de notifications par e-mail.

5.5 Test

Décrivez le processus de test pour le Sprint 3.